

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №5

По дисциплине «Проектирование и реализация баз данных»

Тема: процедуры, функции, триггеры в PostgreSQL

Выполнил Гриценко Я.А.; К3241.

Проверили Говорова М.М., Белов А.О.

Дата 14.06.2026

Цель работы:

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

- Создать 3 процедуры для индивидуальной БД согласно варианту
- Создать 7 триггеров по ИЗ.

Ход работы

Процедуры

Задание 4.

Процедура №1: Получение расписания занятий для групп на определенный день недели

```
CREATE OR REPLACE PROCEDURE public.get_group_schedule_by_day(  
    IN p_group_number VARCHAR,  
    IN p_day_of_week INT,  
    INOUT ref REFCURSOR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    OPEN ref FOR  
        SELECT  
            sg.group_number,  
            l.lesson_date::DATE AS lesson_date,  
            lt.name AS lesson_type,  
            c.room_number,  
            loc.name AS location,  
            t.last_name || ' ' || t.first_name AS teacher_name,  
            d.description AS discipline  
        FROM lesson l  
        JOIN classroom c ON c.id = l.classroom_id  
        JOIN location loc ON loc.id = c.location_id  
        JOIN lesson_type lt ON lt.id = l.lesson_type_id  
        JOIN group_lesson_teacher glt ON glt.lesson_id = l.id  
        JOIN teacher t ON t.id = glt.teacher_id  
        JOIN student_group sg ON sg.id = glt.group_id  
        LEFT JOIN discipline d ON d.id = l.discipline_id  
        WHERE sg.group_number = p_group_number  
        AND EXTRACT(DOW FROM l.lesson_date) = p_day_of_week  
        ORDER BY l.lesson_date;  
END;  
$$;
```

Вызовем процедуру и проверим результат ее работы:

```

pin_db=# BEGIN;
BEGIN
pin_db=# CALL get_group_schedule_by_day('A5-25', 1, 'cur');
ref
-----
cur
(1 строка)

pin_db=# FETCH ALL IN cur;
group_number | lesson_date | lesson_type | room_number | location | teacher_name | discipline
-----
A5-25 | 2025-12-01 | лекция | 241 | Башкирэнерго | Бобылев Геннадий | Сети - учебная дисциплина
A5-25 | 2025-12-08 | лекция | 215 | Башкирэнерго | Бобылев Геннадий | Мобильная разработка - учебная дисциплина
A5-25 | 2025-12-22 | практическое занятие | 273 | Карпов Инкорпорэйтэд | Бобылев Геннадий | Сети - учебная дисциплина
A5-25 | 2026-01-19 | лекция | 249 | ОАО «Казакова Шарова» | Бобылев Геннадий | Мобильная разработка - учебная дисциплина
A5-25 | 2026-01-19 | практическое занятие | 273 | Карпов Инкорпорэйтэд | Бобылев Геннадий | Мобильная разработка - учебная дисциплина
A5-25 | 2026-04-13 | лекция | 317 | Бурова и партнеры | Прохоров Никодим | Базы данных - учебная дисциплина
(6 строк)

pin_db=# COMMIT;

```

2. Запись на курс слушателя.

CREATE OR REPLACE PROCEDURE enroll_student(

p_student_id **BIGINT**,

p_group_id **BIGINT**

)

LANGUAGE plpgsql

AS \$\$

DECLARE

v_current_count **INT**;

v_max_students **INT**;

v_group_start **DATE**;

v_group_end **DATE**;

v_already_enrolled **INT**;

BEGIN

-- Проверка: существует ли группа и получаем её лимиты

SELECT max_students, start_date, end_date

INTO v_max_students, v_group_start, v_group_end

FROM student_group

WHERE id = p_group_id;

IF NOT FOUND THEN

RAISE NOTICE 'Ошибка: группа с ID % не найдена', p_group_id;

RETURN;

END IF;

-- Проверка: существует ли студент

IF NOT EXISTS (SELECT 1 FROM student WHERE id = p_student_id) THEN

RAISE NOTICE 'Ошибка: студент с ID % не найден', p_student_id;

RETURN;

END IF;

-- Проверка: не записан ли уже

SELECT COUNT(*) INTO v_already_enrolled

FROM student_in_group

WHERE student_id = p_student_id **AND** group_id = p_group_id;

IF v_already_enrolled > 0 **THEN**

RAISE NOTICE 'Ошибка: студент уже записан в эту группу';

RETURN;

END IF;

-- Проверка: не превышен ли лимит

SELECT COUNT(*) INTO v_current_count

FROM student_in_group

WHERE group_id = p_group_id;

IF v_current_count >= v_max_students **THEN**

RAISE NOTICE 'Ошибка: группа заполнена (максимум: %)', v_max_students;

RETURN;

END IF;

-- Запись студента

INSERT INTO student_in_group (student_id, group_id, start_date, end_date)

VALUES (p_student_id, p_group_id, v_group_start, v_group_end);

RAISE NOTICE 'Студент % успешно записан в группу %', p_student_id, p_group_id;

END;

\$\$;

Добавим студента в группу, проверим, что он добавился, проверим, что повторное добавление дает ошибку.

```
pin_db=# CALL enroll_student(1, 2);
ЗАМЕЧАНИЕ: Студент 1 успешно записан в группу 2
CALL
pin_db=# SELECT * FROM student_in_group WHERE student_id = 1 AND group_id = 2;
 student_id | group_id | start_date | end_date
-----+-----+-----+-----
          1 |         2 | 2026-03-11 | 2026-08-30
(1 строка)

pin_db=# CALL enroll_student(1, 2);
ЗАМЕЧАНИЕ: Ошибка: студент уже записан в эту группу
CALL
```

3. Получения перечня свободных лекционных аудиторий на любой день недели

```
CREATE OR REPLACE PROCEDURE get_free_lecture_rooms(  
    IN p_day_of_week INT,  
    INOUT ref REFCURSOR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    OPEN ref FOR  
        SELECT c.id, c.room_number, loc.name AS location, loc.address  
        FROM classroom c  
        JOIN location loc ON loc.id = c.location_id  
        WHERE c.type = 'лекционная'  
        AND c.id NOT IN (  
            SELECT l.classroom_id  
            FROM lesson l  
            WHERE EXTRACT(DOW FROM l.lesson_date) = p_day_of_week  
        )  
        ORDER BY loc.name, c.room_number;  
END;  
$$;
```

```
pin_db=# BEGIN;  
BEGIN  
pin_db=*# CALL get_free_lecture_rooms(1, 'cur');  
ref  
-----  
cur  
(1 строка)  
  
pin_db=*# FETCH ALL IN cur;  
id | room_number | location | address  
---+-----+-----+-----  
13 | 252 | Корпус на Гагарина | ул. Гагарина, 5  
10 | 434 | Корпус на Гагарина | ул. Гагарина, 5  
12 | 364 | Корпус на Ленина | ул. Ленина, 10  
16 | 372 | Корпус на Ленина | ул. Ленина, 10  
(4 строки)
```

Триггеры

1. Проверка лимита студентов в группе

Создадим функцию

```
CREATE OR REPLACE FUNCTION trg_check_max_students()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
DECLARE
```

```
    v_current INT;
```

```
    v_max INT;
```

```
BEGIN
```

```
    SELECT COUNT(*), sg.max_students
```

```
    INTO v_current, v_max
```

```
    FROM student_in_group sig
```

```
    JOIN student_group sg ON sg.id = sig.group_id
```

```
    WHERE sig.group_id = NEW.group_id
```

```
    GROUP BY sg.max_students;
```

```
    IF v_current >= v_max THEN
```

```
        RAISE EXCEPTION 'Группа заполнена (максимум: %)', v_max;
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```


\$\$;

Создадим триггер

```
CREATE TRIGGER trg_check_max_students  
BEFORE INSERT ON student_in_group  
FOR EACH ROW  
EXECUTE FUNCTION trg_check_max_students();
```

Проверим:

```
pin_db=# INSERT INTO student_in_group (student_id, group_id, start_date, end_date)  
pin_db=# VALUES (1, 13, '2025-01-01', '2025-06-01');  
ОШИБКА: Группа заполнена (максимум: 14)  
КОНТЕКСТ: функция PL/pgSQL trg_check_max_students(), строка 14, оператор RAISE
```

2. Логирование изменений расписания

Таблица логов:

```
CREATE TABLE lesson_audit (  
    id SERIAL PRIMARY KEY,  
    lesson_id BIGINT NOT NULL,  
    changed_at TIMESTAMP DEFAULT NOW(),  
    old_date TIMESTAMP,  
    new_date TIMESTAMP,  
    old_classroom_id BIGINT,  
    new_classroom_id BIGINT  
);
```

Функция триггера:

```
CREATE OR REPLACE FUNCTION trg_lesson_audit()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$
```

BEGIN

INSERT INTO lesson_audit (lesson_id, old_date, new_date, old_classroom_id,
new_classroom_id)

VALUES (**OLD**.id, **OLD**.lesson_date, **NEW**.lesson_date, **OLD**.classroom_id,
NEW.classroom_id);

RETURN NEW;

END;

\$\$;

Создание триггера

CREATE TRIGGER trg_lesson_audit

AFTER UPDATE ON lesson

FOR EACH ROW

EXECUTE FUNCTION trg_lesson_audit();

Проверим триггер:

```
pin_db=# UPDATE lesson
pin_db=# SET lesson_date = '2026-05-09 10:00:00'
pin_db=# WHERE id = 1;
UPDATE 1
pin_db=# SELECT * FROM lesson_audit;
 id | lesson_id |      changed_at      | old_date | new_date | old_classroom_id | new_classroom_id
-----+-----+-----+-----+-----+-----+-----
  1 |         1 | 2026-06-14 14:16:51.321573 | 2026-05-09 10:00:00 | 2026-05-09 10:00:00 |          26 |          26
(1 строка)
```

3. Проверка дат студента в группе

Таблица логов:

CREATE OR REPLACE FUNCTION trg_check_student_dates()

RETURNS TRIGGER

LANGUAGE plpgsql

AS \$\$

DECLARE

v_group_start **DATE**;

v_group_end **DATE**;

BEGIN

SELECT start_date, end_date

INTO v_group_start, v_group_end

FROM student_group

WHERE id = **NEW**.group_id;

```

IF NEW.start_date < v_group_start OR NEW.end_date > v_group_end THEN
    RAISE EXCEPTION 'Даты студента (%) выходят за рамки дат группы (% - %)',
        NEW.start_date || ' - ' || NEW.end_date,
        v_group_start, v_group_end;
END IF;

RETURN NEW;
END;
$$;

```

Создание триггера

```

CREATE TRIGGER trg_check_student_dates
BEFORE INSERT OR UPDATE ON student_in_group
FOR EACH ROW
EXECUTE FUNCTION trg_check_student_dates();

```

Проверим триггер:

```

pin_db=# INSERT INTO student_in_group (student_id, group_id, start_date, end_date)
pin_db=# VALUES (1, 1, '2025-09-01', '2026-01-01');
ОШИБКА: Даты студента (2025-09-01 - 2026-01-01) выходят за рамки дат группы (2025-10-20 - 2025-12-19)
КОНТЕКСТ: функция PL/pgSQL trg_check_student_dates(), строка 12, оператор RAISE

```

4. Проверка преподавателя на право вести дисциплину

```

CREATE OR REPLACE FUNCTION trg_check_teacher_discipline()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_discipline_id BIGINT;
BEGIN
    SELECT discipline_id INTO v_discipline_id
    FROM lesson
    WHERE id = NEW.lesson_id;

    IF v_discipline_id IS NOT NULL AND NOT EXISTS (
        SELECT 1 FROM teacher_discipline
        WHERE teacher_id = NEW.teacher_id AND discipline_id = v_discipline_id
    ) THEN
        RAISE EXCEPTION 'Преподаватель % не может вести эту дисциплину',
        NEW.teacher_id;
    END IF;

    RETURN NEW;
END;

```

\$\$;

Создание триггера

```
CREATE TRIGGER trg_check_teacher_discipline  
BEFORE INSERT ON group_lesson_teacher  
FOR EACH ROW  
EXECUTE FUNCTION trg_check_teacher_discipline();
```

Проверим триггер:

```
pin_db=# INSERT INTO group_lesson_teacher (group_id, lesson_id, teacher_id)  
pin_db=# VALUES (14, 1, 1);  
ОШИБКА: Преподаватель 1 не может вести эту дисциплину  
КОНТЕКСТ: функция PL/pgSQL trg_check_teacher_discipline(), строка 13, оператор RAISE
```

5. Автозаполнение дат студента

Если при записи студента не указаны даты - подставляет даты группы.

Таблица логов:

```
CREATE OR REPLACE FUNCTION trg_auto_fill_dates()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  IF NEW.start_date IS NULL OR NEW.end_date IS NULL THEN  
    SELECT start_date, end_date  
    INTO NEW.start_date, NEW.end_date  
    FROM student_group  
    WHERE id = NEW.group_id;  
  END IF;  
  
  RETURN NEW;  
END;  
$$;
```

Создание триггера

```
CREATE TRIGGER trg_auto_fill_dates  
BEFORE INSERT ON student_in_group  
FOR EACH ROW  
EXECUTE FUNCTION trg_auto_fill_dates();
```

Проверим триггер:

```

pin_db=# INSERT INTO group_lesson_teacher (group_id, lesson_id, teacher_id)
pin_db=# VALUES (14, 1, 1);
ОШИБКА: Преподаватель 1 не может вести эту дисциплину
КОНТЕКСТ: функция PL/pgSQL trg_check_teacher_discipline(), строка 13, оператор RAISE
pin_db=#
pin_db=#
pin_db=#
pin_db=# INSERT INTO student_in_group (student_id, group_id) VALUES (2, 1);
INSERT 0 1
pin_db=# SELECT * FROM student_in_group WHERE student_id = 2 AND group_id = 1;
 student_id | group_id | start_date | end_date
-----+-----+-----+-----
          2 |         1 | 2025-10-20 | 2025-12-19
(1 строка)

```

6. Запрет удаления группы, в которой есть студенты

Функция триггера:

```

CREATE OR REPLACE FUNCTION trg_prevent_group_delete()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM student_in_group
    WHERE group_id = OLD.id;

    IF v_count > 0 THEN
        RAISE EXCEPTION 'Нельзя удалить группу, в которой есть студенты (сейчас: %)',
v_count;
    END IF;

    RETURN OLD;
END;
$$;

```

Создание триггера

```

CREATE TRIGGER trg_prevent_group_delete
BEFORE DELETE ON student_group
FOR EACH ROW
EXECUTE FUNCTION trg_prevent_group_delete();

```

Проверим триггер:

```
pin_db=# DELETE FROM student_group WHERE id = 1;
ОШИБКА: Нельзя удалить группу, в которой есть студенты (сейчас: 13)
КОНТЕКСТ: функция PL/pgSQL trg_prevent_group_delete(), строка 10, оператор RAISE
```

7. Запрет вставки занятия на занятую аудиторию в то же время

```
CREATE OR REPLACE FUNCTION trg_check_classroom_busy()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM lesson
        WHERE classroom_id = NEW.classroom_id
        AND lesson_date = NEW.lesson_date
        AND id != NEW.id
    ) THEN
        RAISE EXCEPTION 'Аудитория занята в это время';
    END IF;

    RETURN NEW;
END;
$;
```

Создание триггера

```
CREATE TRIGGER trg_check_classroom_busy
BEFORE INSERT OR UPDATE ON lesson
FOR EACH ROW
EXECUTE FUNCTION trg_check_classroom_busy();
```

Проверим триггер:

```
pin_db=# INSERT INTO lesson (classroom_id, lesson_type_id, lesson_date)
pin_db=# VALUES (26, 1, '2026-05-09 10:00:00');
ОШИБКА: Аудитория занята в это время
КОНТЕКСТ: функция PL/pgSQL trg_check_classroom_busy(), строка 9, оператор RAISE
```

Выводы

В ходе работы были изучены и применены на практике хранимые процедуры и триггеры в СУБД PostgreSQL для базы данных «Курсы». Разработано три процедуры: получение расписания группы по дню недели, запись слушателя на курс с проверками, поиск свободных лекционных аудиторий. Создано семь триггеров, автоматизирующих контроль целостности данных: проверка лимита студентов в группе, контроль дат обучения, проверка соответствия преподавателя дисциплине, автозаполнение дат, логирование изменений расписания, запрет удаления непустой группы, проверка занятости аудитории. Проверка работоспособности выполнена тестовыми запросами — все объекты функционируют корректно.